

软件详细设计说明书

文档标识: SDD-DETAIL-LIFE-ASSIST-2026-V1.0 版本号: V1.0 日期: 2026年

目录

软件详细设计说明书	
1 引言	
1.1 编写目的	
1.2 背景	
1.3 术语表	
1.4 参考文档	
2 系统结构设计及子系统划分	
2.1 子系统划分	
2.2 子系统活动图	
SYS-01 用户认证子系统	
SYS-04 订单与支付子系统	
2.3 子系统顺序图	
外卖下单与支付顺序图	
商家核销券码顺序图	
3 功能模块设计	
3.1 模块总览	
3.2 模块详细设计	
M-01 用户认证模块	
M-02 用户信息模块	
M-03 商家浏览模块	
M-04 购物车模块	
M-05 订单管理模块	
M-06 支付模块	
M-07 券码模块	
M-08 评价模块	
M-09 商家管理模块	
M-10 平台管理模块	
M-11 AI 客服模块	
4 界面设计	

4.1 外部界面设计	
4.1.1 前后端接口规范	
4.1.2 完整 API 接口清单	
4.2 内部界面设计	
4.2.1 后端模块间调用关系	
4.2.2 前端组件调用关系	
4.3 用户界面设计	
4.3.1 设计规范	
4.3.2 页面详细设计	
4.3.3 错误提示设计	
5 异常处理设计	
5.1 出错信息管理	
5.1.1 后端异常分类与处理	
5.1.2 前端错误处理	
5.2 故障预防与补救	
5.2.1 支付幂等性保障	
5.2.2 订单事务一致性	
5.2.3 数据完整性	
5.2.4 状态机保护	
5.3 系统维护设计	
5.3.1 健康检查	
5.3.2 操作日志	
5.3.3 数据重置	
6 性能优化和安全设计	
6.1 性能优化设计	
6.1.1 前端优化	
6.1.2 后端优化	
6.1.3 后端性能指标实测估计	
6.2 安全设计	
6.2.1 认证与鉴权	
6.2.2 权限控制详情	
6.2.3 数据安全	
6.2.4 Spring Security 配置	
7 项目测试计划	
7.1 测试策略	
7.2 测试类型与范围	

7.3 后端单元测试用例

7.4 集成测试计划

7.5 系统测试计划

7.6 安全测试计划

1 引言

1.1 编写目的

本文档是《综合生活助手平台软件概要设计说明书》的补充文档，旨在对系统的子系统划分、功能模块、界面设计、异常处理、性能安全及测试计划等做出详细设计说明，为编码实现、单元测试和系统集成提供直接依据。

本文档的预期读者包括项目开发人员（用于指导编码实现）、测试人员（用于设计测试用例）以及课程指导教师（用于评审设计完整性）。

1.2 背景

待开发软件系统名称为“综合生活助手平台”（LumaLife）。本平台是一款面向城市居民和本地商家的生活服务类 Web/H5 综合应用，主要覆盖商家搜索与推荐、外卖点餐、到店团购、用户评价、商家后台管理和模拟支付等业务。

本项目为课程实验项目，由项目小组负责开发。项目组成员包括孙伟宸、黄羿、孙铭、钟其宏、戴昊。系统最终运行于本地测试环境，客户端通过现代浏览器访问，数据采用内存存储，不依赖外部数据库。

本系统的主要用户包括：

- 普通用户（USER）：浏览商家、管理购物车、创建订单、模拟支付、查看订单状态、提交评价；
- 商家管理员（MERCHANT_ADMIN）：维护商家和商品信息、处理订单状态流转、核销团购券码；
- 平台管理员（PLATFORM_ADMIN）：查看平台核心指标和操作日志。

1.3 术语表

术语	定义
SPA	Single Page Application，单页应用，前端在一个 HTML 页面内通过状态切换实现多视图
JWT	JSON Web Token，本系统采用简化 Bearer Token 方案，由服务端生成 UUID 映射用户身份
BCrypt	密码哈希算法，用于用户密码的安全存储
DTO	Data Transfer Object，数据传输对象，用于封装请求参数
幂等性	同一操作执行一次或多次产生相同结果的性质，本系统支付接口通过 clientRequestId 保证幂等
状态机	订单状态的有限状态集合及合法流转规则
软删除	通过标记字段而非物理删除来标记数据失效
Cents	本系统金额统一以"分"为单位存储的整数表示法
Vite	前端构建工具，提供开发服务器和热更新
CSS Variables	CSS 自定义属性，本系统用于全局主题色管理

1.4 参考文档

编号	文件或资料	版本/日期
[1]	软件概要设计说明书（SDD-LIFE-ASSIST-2026-V1.0）	V1.0，2026年
[2]	软件需求规格说明书（SRS-LIFE-ASSIST-2026-V1.0）	V1.0，2026年
[3]	软件开发计划书（SDP-LIFE-ASSIST-2026-V1.0）	V1.0，2026年
[4]	GB/T 8567-2006，计算机软件文档编制规范	2006年
[5]	Spring Boot 3.3 Reference Documentation	—
[6]	React 18 Documentation	—

2 系统结构设计及子系统划分

2.1 子系统划分

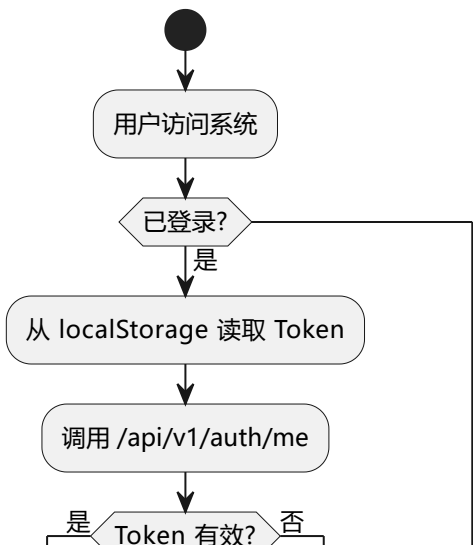
综合生活助手平台按业务域划分为 8 个子系统，各子系统对应概要设计中的功能模块：

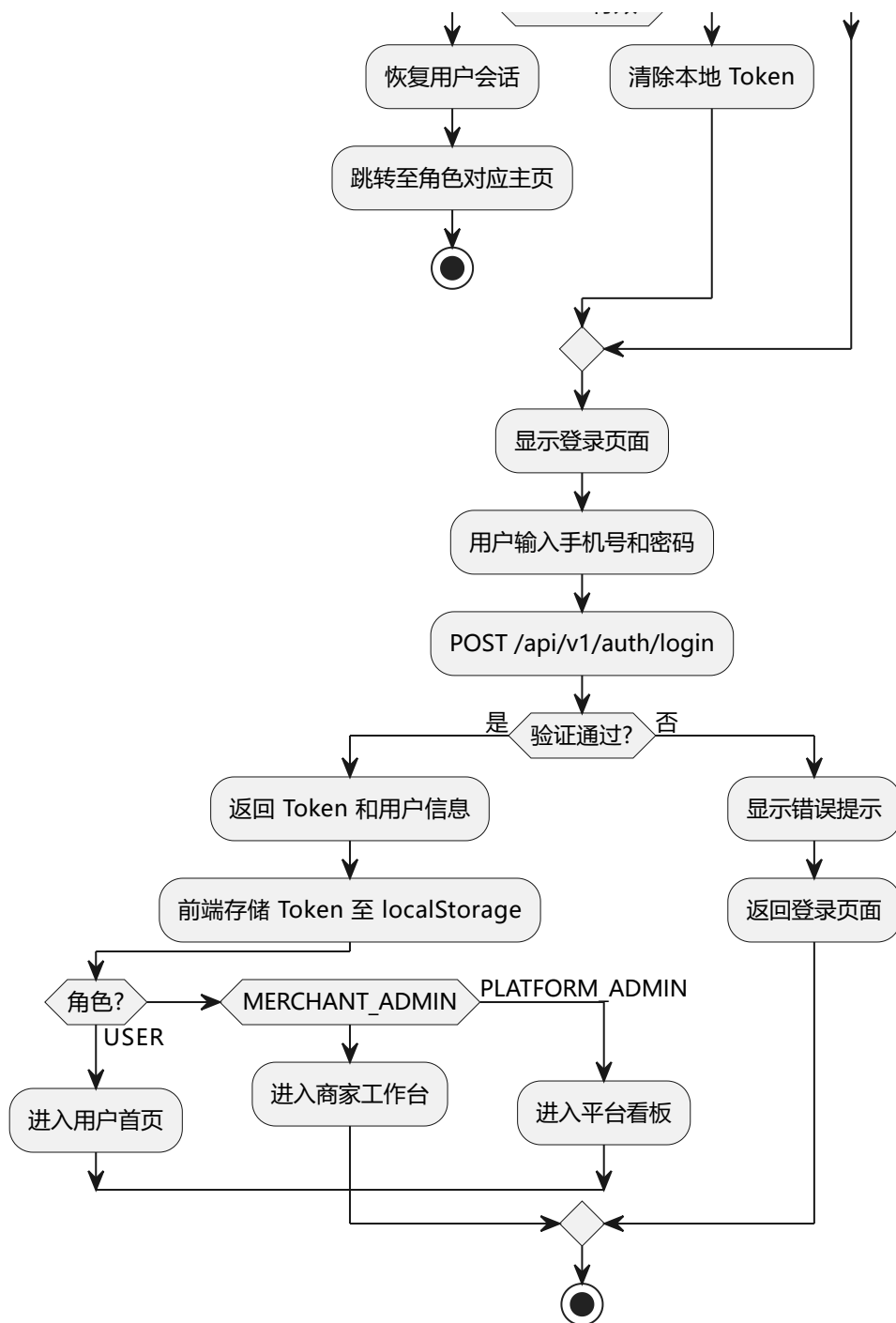
子系统编号	子系统名称	功能概述	对应需求
SYS-01	用户认证子系统	注册、登录、Token 鉴权、个人信息维护、地址管理	CR-01
SYS-02	商家浏览子系统	分类筛选、关键词搜索、推荐排序、商家详情展示	CR-02
SYS-03	购物车子系统	加购、修改数量、删除/清空购物车	CR-03
SYS-04	订单与支付子系统	外卖下单、团购下单、模拟支付、订单取消、状态跟踪	CR-03, CR-04
SYS-05	团购与券码子系统	团购套餐管理、券码生成、商家核销	CR-04
SYS-06	评价子系统	订单评价、评分统计、重复评价限制	CR-05
SYS-07	商家后台子系统	商家工作台、订单处理、商品/套餐管理	CR-06
SYS-08	平台管理子系统	平台指标看板、操作日志、健康检查	CR-06

2.2 子系统活动图

SYS-01 用户认证子系统

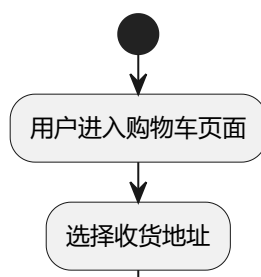
用户认证子系统活动图

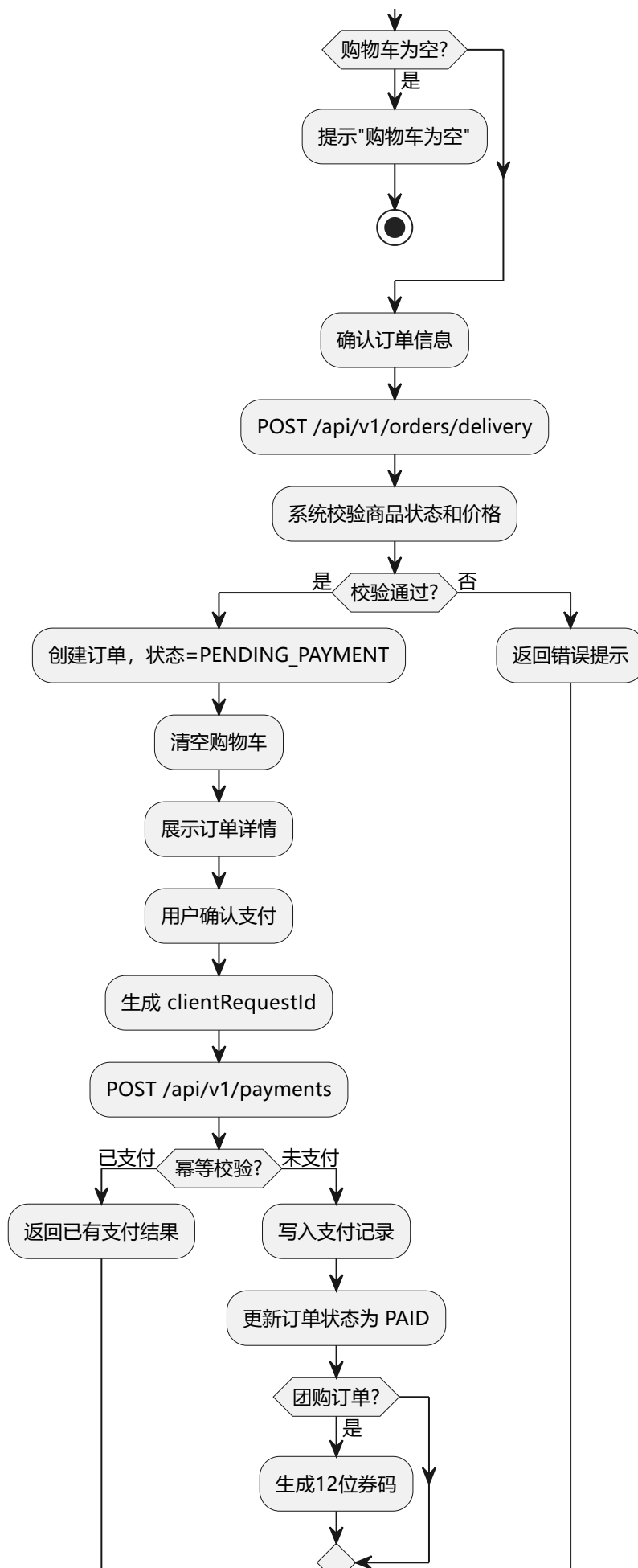


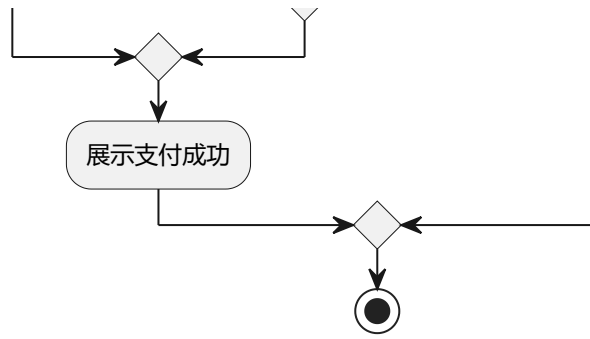


SYS-04 订单与支付子系统

外卖下单与模拟支付活动图



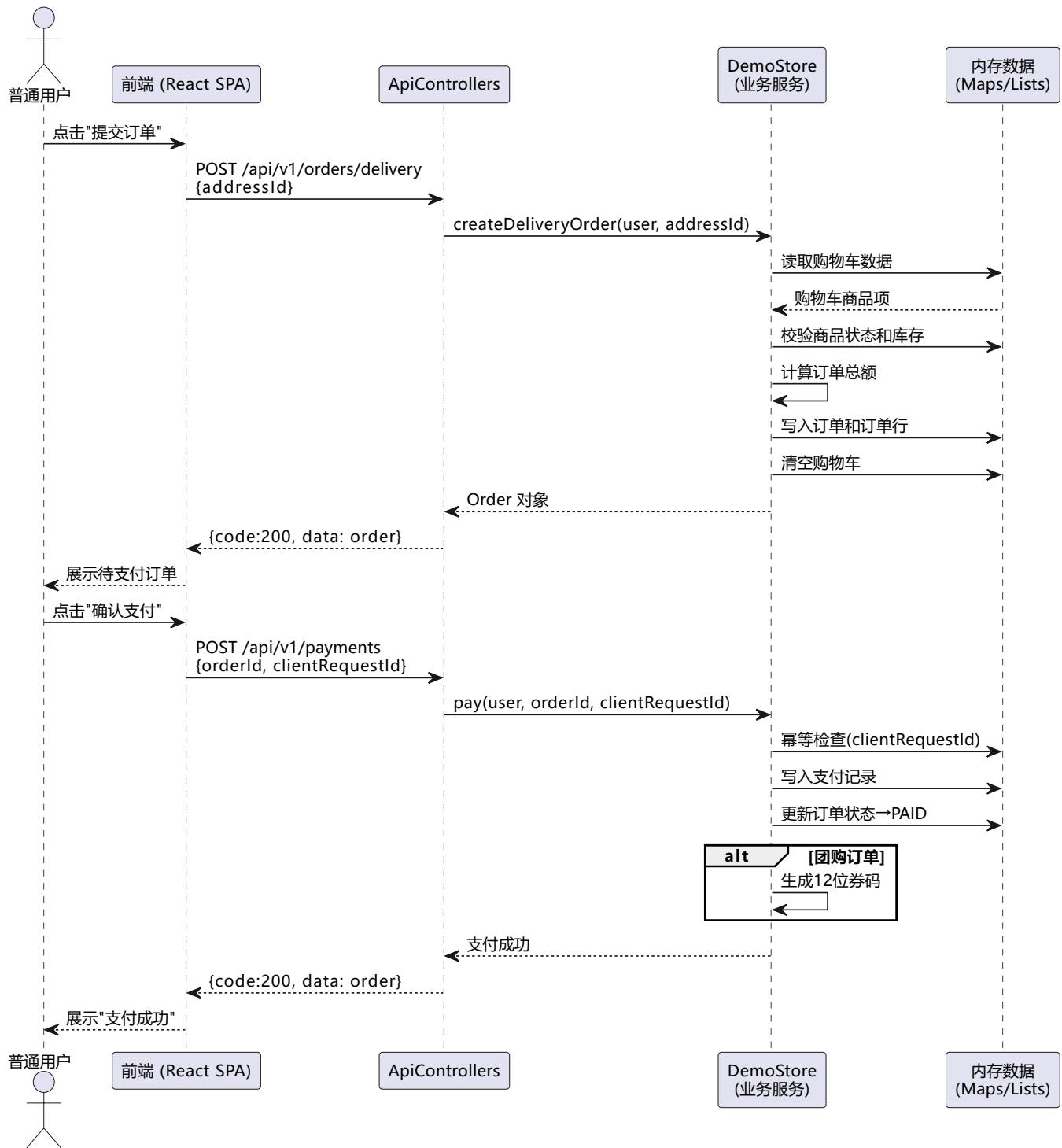




2.3 子系统顺序图

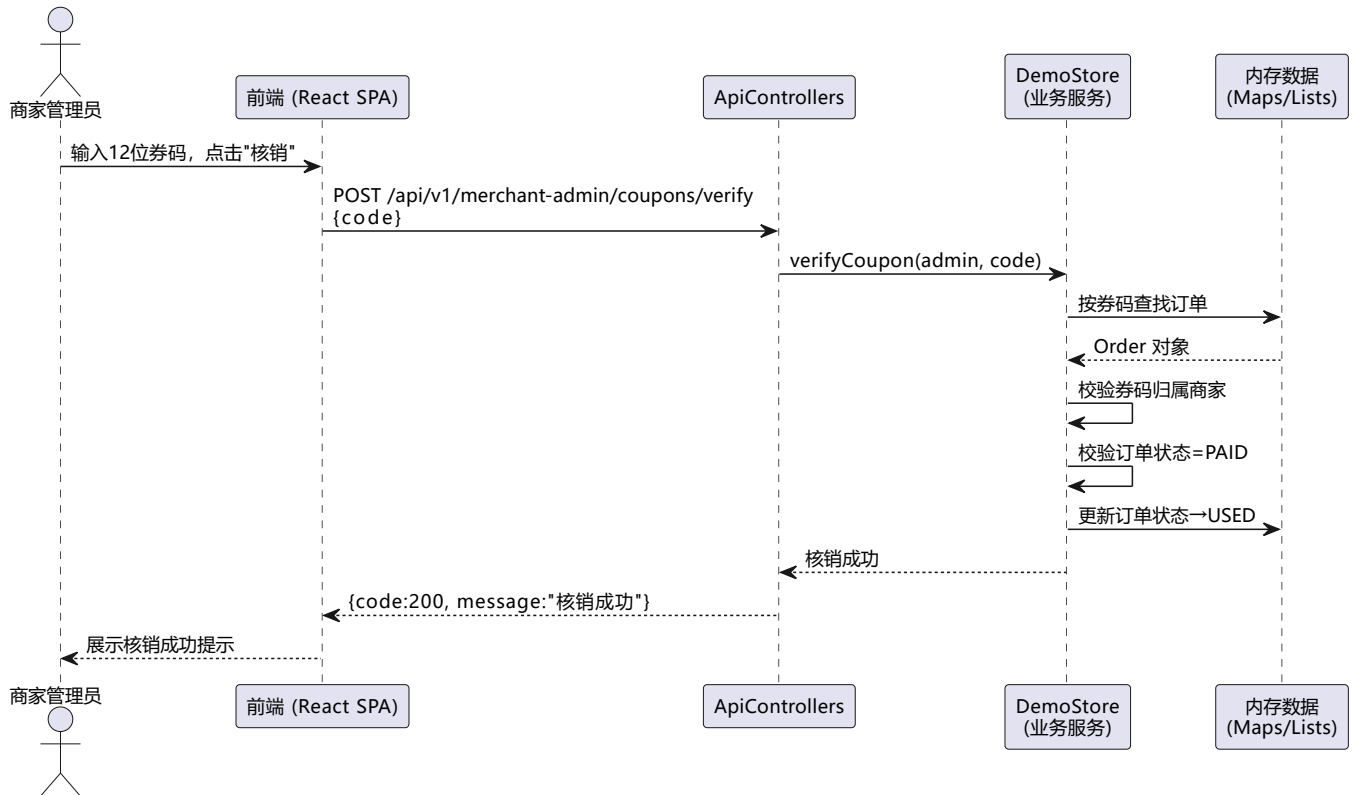
外卖下单与支付顺序图

外卖下单与模拟支付顺序图



商家核销券码顺序图

商家核销团购券码顺序图



3 功能模块设计

3.1 模块总览

系统后端采用单控制器 + 单业务服务的紧凑架构。所有 REST 接口集中在 `ApiControllers` 类中，所有业务逻辑集中在 `DemoStore` 服务类中。模块按业务功能组织，而非按物理文件划分。

模块编号	模块名称	对应子系统	后端核心方法
M-01	用户认证模块	SYS-01	login, register, userByToken
M-02	用户信息模块	SYS-01	updateProfile, addresses, saveAddress, deleteAddress
M-03	商家浏览模块	SYS-02	categories, merchants, merchantDetail
M-04	购物车模块	SYS-03	cartDetail, addCart, updateCartItem, removeCartItem, clearCart
M-05	订单管理模块	SYS-04	createDeliveryOrder, createGroupOrder, userOrders, cancel
M-06	支付模块	SYS-04	pay
M-07	券码模块	SYS-05	verifyCoupon
M-08	评价模块	SYS-06	review
M-09	商家管理模块	SYS-07	merchantOrders, transition, merchantProducts, saveProduct, toggleProduct, merchantDeals, saveDeal, toggleDeal
M-10	平台管理模块	SYS-08	adminMetrics

模块编号	模块名称	对应子系统	后端核心方法
M-11	AI 客服模块	加分项	askAssistant

3.2 模块详细设计

M-01 用户认证模块

项目	内容
模块编号	M-01
模块名称	用户认证模块
输入	LoginRequest{phone, password} 或 RegisterRequest{phone, password, nickname}
处理	1. 登录：查询 phone 对应的 User，BCrypt 比对密码，生成 UUID Token，返回 Token+User；2. 注册：校验手机号唯一性，BCrypt 编码密码，创建 User 记录，自动登录；3. Token 验证：从 Authorization Header 提取 Bearer Token，查 Map<String,Long> 获取 userId
算法描述	密码编码采用 BCryptPasswordEncoder(rounds=10)。Token 采用 UUID.randomUUID().toString() 生成，存入 ConcurrentHashMap<String, Long>，键为 Token，值为 userId。前端将 Token 存入 localStorage，每次请求通过 Authorization: Bearer 携带
输出	成功返回 {code:200, data: {token, user}}；失败返回 {code:40100, message:"密码错误"} 等

M-02 用户信息模块

项目	内容
模块编号	M-02
模块名称	用户信息模块
输入	ProfileRequest{nickname, avatarUrl}、AddressRequest{id, contactName, phone, detail, defaultAddress}
处理	1. 更新昵称/头像：创建新 User record 替换旧对象（不可变更新）；2. 查询地址列表：按 userId 过滤；3. 新增地址：校验数量上限（5个），若为首个地址则自动设为默认；4. 设为默认：遍历该用户所有地址，仅保留目标为默认；5. 删除地址：从列表中移除
算法描述	地址上限控制： <code>if (list.size() >= 5) throw BusinessException(40000, "最多添加5个地址")</code> 。默认地址互斥：设置默认时将所有其他地址的 defaultAddress 置为 false
输出	地址列表返回 List ， 更新操作返回更新后的 User 对象（已脱敏）

M-03 商家浏览模块

项目	内容
模块编号	M-03
模块名称	商家浏览模块
输入	查询参数: keyword(可选), categoryId(可选), sort(可选: "distance"/"recommend"/"score"), page(默认1), size(默认10)
处理	1. 分类列表: 返回全部 Category; 2. 商家列表: 按关键词(name/address 包含)、分类 ID 过滤; 按距离/推荐度/评分排序; 分页返回; 3. 商家详情: 查询商家信息 + 在售商品 + 有效团购套餐 + 评价列表
算法描述	关键词过滤: <code>m.name().contains(keyword) m.address().contains(keyword)</code> 。排序策略: distance 按 distanceKm 升序; score 按 avgScore 降序; recommend 按综合推荐度 (评分权重 0.7 + 距离权重 0.3) 降序。分页通过 PageResponse.of() 计算总页数
输出	商家列表返回 PageResponse; 商家详情返回 {merchant, products, deals, reviews}

M-04 购物车模块

项目	内容
模块编号	M-04
模块名称	购物车模块
输入	CartRequest{productId, quantity}、CartQuantityRequest{quantity}
处理	1. 查看购物车：按 userId 从 Map 读取 CartItem 列表，联查 Product 信息生成 CartLine（含小计）；2. 加入购物车：校验商品存在且在售，校验同一商家约束（购物车中已有其他商家商品时清空重加），相同商品则累加数量；3. 修改数量：更新 quantity，quantity<=0 时删除该项；4. 删除单项：按 productId 移除；5. 清空购物车：删除该用户的整个购物车记录
算法描述	同商家约束： <code>if (!cart.isEmpty() && cart.get(0).merchantId != newMerchantId) { cart.clear(); }</code> 。CartLine 小计计算： <code>subtotalCent = priceCent * quantity</code> 。购物车数据存储在 <code>Map<Long, List<CartItem>></code> （键为 userId）
输出	CartLine 列表，每项含 productId, merchantId, name, priceCent, quantity, subtotalCent

M-05 订单管理模块

项目	内容
模块编号	M-05
模块名称	订单管理模块
输入	DeliveryOrderRequest{addressId}、GroupOrderRequest{dealId, quantity}
处理	1. 外卖下单：读取购物车详情，校验购物车非空，根据 addressId 查找收货地址并做快照，创建 Order(OrderType=DELIVERY, status=PENDING_PAYMENT)，将购物车商品转为 OrderLine，计算总额，清空购物车；2. 团购下单：查询 GroupDeal 信息，校验有效且在售，创建 Order(OrderType=GROUP_BUY)，计算总额(priceCent * quantity)；3. 查询订单：按 userId 或 merchantId 过滤，按创建时间倒序；4. 取消订单：仅 PENDING_PAYMENT 状态可取消
算法描述	订单号生成： "DLV" + System.currentTimeMillis() + userId 。状态时间线记录：每次状态变更在 statusTimeline Map 中记录 <status, createdAt> 。地址快照：order.addressSnapshot = address.toString()。订单总额：所有 OrderLine.priceCent * quantity 求和
输出	Order 对象，含订单号、状态、商品行、金额、地址快照、状态时间线

M-06 支付模块

项目	内容
模块编号	M-06
模块名称	模拟支付模块
输入	PaymentRequest{orderId, clientId}
处理	1. 校验订单归属当前用户；2. 校验订单状态为 PENDING_PAYMENT；3. 幂等检查：按 clientId 查是否已有支付记录，如有则直接返回已有结果；4. 创建 Payment 记录(PaymentStatus=SUCCESS)；5. 更新订单状态为 PAID；6. 若为团购订单，生成12位数字券码
算法描述	幂等控制： Map<String, Long> paymentIdempotent 存储 clientId → paymentId 映射。券码生成： String.format("%012d", ThreadLocalRandom.current().nextLong(0, 999999999999L))。支付记录包含 paymentId, orderId, amount, status, paidAt
输出	更新后的 Order 对象（含券码，如适用）

M-07 券码模块

项目	内容
模块编号	M-07
模块名称	团购券码核销模块
输入	VerifyRequest{code}
处理	1. 查询券码对应订单；2. 校验订单归属当前商家管理员；3. 校验订单类型为 GROUP_BUY 且状态为 PAID；4. 更新订单状态为 USED，记录核销时间
算法描述	券码查找： <code>orders.values().filter(o -> code.equals(o.couponCode)).findFirst()</code> 。归属校验： <code>o.merchantId.equals(admin.merchantId)</code>
输出	{code:200, message:"核销成功"} 或对应错误码

M-08 评价模块

项目	内容
模块编号	M-08
模块名称	评价模块
输入	ReviewRequest{orderId, overallScore, tasteScore, serviceScore, content}
处理	1. 校验订单归属当前用户；2. 校验订单状态为 COMPLETED 或 USED（已完成/已使用）；3. 校验未重复评价（order.reviewed == false）；4. 创建 Review 记录；5. 更新订单 reviewed=true；6. 重新计算商家平均评分
算法描述	评分范围校验：各评分 1-5 分。内容截断： <code>content.substring(0, Math.min(content.length(), 200))</code> 。商家评分重算：收集该商家所有评价的各维度分数，计算算术平均值，更新 merchant.avgScore
输出	{code:200, data: review, message:"评价成功"}

M-09 商家管理模块

项目	内容
模块编号	M-09
模块名称	商家管理模块
输入	TransitionRequest{nextStatus}、ProductRequest{id, name, description, priceCent, stock}、DealRequest{id, title, description, priceCent, stock}
处理	1. 订单处理：校验订单归属当前商家，按状态机规则推进（PAID→ACCEPTED→DELIVERING→COMPLETED），记录状态时间线；2. 商品管理：新增或更新 Product，toggleProduct 切换 listed 状态；3. 团购管理：新增或更新 GroupDeal，toggleDeal 切换 active 状态
算法描述	状态机：if (current==PAID && next==ACCEPTED) ...; else if (current==ACCEPTED && next==DELIVERING) ...; else if (current==DELIVERING && next==COMPLETED) ...; else throw BusinessException。商品ID自增：productId.incrementAndGet()。上架/下架切换：product.listed = !product.listed
输出	操作结果或更新后的对象

M-10 平台管理模块

项目	内容
模块编号	M-10
模块名称	平台管理模块
输入	无（GET 请求）
处理	统计用户总数、商家总数、订单总数、交易总额（已支付订单金额求和），收集操作日志
算法描述	交易总额： <code>orders.values().stream().filter(o -> o.status != PENDING_PAYMENT && o.status != CANCELLED).mapToLong(Order::totalCent).sum()</code> 。用户数按 <code>role==USER</code> 统计
输出	<code>{userCount, merchantCount, orderCount, totalRevenueCents, health:"UP", logs: List}</code>

M-11 AI 客服模块

项目	内容
模块编号	M-11
模块名称	AI 客服模块（规则型）
输入	AskRequest{question}
处理	关键词匹配，返回预定义的常见问题答案
算法描述	关键词映射：包含"支付"→返回支付说明；包含"取消"→返回取消规则；包含"评价"→返回评价条件；包含"券码"/"核销"→返回券码说明；包含"团购"→返回团购流程；默认返回兜底引导语
输出	{code:200, data: {answer: "..."}}}

4 界面设计

4.1 外部界面设计

4.1.1 前后端接口规范

前端与后端通过 HTTP/JSON 通信，接口基础路径为 `/api/v1`。所有响应采用统一格式：

```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

请求头规范：

- Content-Type: application/json (所有 POST 请求)
- Authorization: Bearer <token> (需认证的接口)

4.1.2 完整 API 接口清单

认证接口（无需鉴权）：

方法	路径	请求体	响应 data	说明
POST	/api/v1/auth/login	{phone, password}	{token, user}	用户登录
POST	/api/v1/auth/register	{phone, password, nickname}	{token, user}	用户注册
GET	/api/v1/auth/me	—	User	获取当前用户信息

公开浏览接口（无需鉴权）：

方法	路径	查询参数	响应 data	说明
GET	/api/v1/categories	—	Category[]	分类列表
GET	/api/v1/merchants	keyword, categoryId, sort, page, size	PageResponse<Merchant>	商家搜索/列表
GET	/api/v1/merchants/{id}	—	{merchant, products, deals, reviews}	商家详情

用户接口（需 USER 角色）：

方法	路径	请求体/参数	响应 data	说明
POST	/api/v1/user/profile	{nickname, avatarUrl}	User	更新个人信息
GET	/api/v1/user/addresses	—	Address[]	查询地址列表
POST	/api/v1/user/addresses	{id, contactName, phone, detail, defaultAddress}	Address	新增/更新地址
POST	/api/v1/user/addresses/{id}/default	—	—	设为默认地址
POST	/api/v1/user/addresses/{id}/delete	—	—	删除地址

购物车接口（需 USER 角色）：

方法	路径	请求体/参数	响应 data	说明
GET	/api/v1/cart	—	CartItem[]	查看购物车（简要）
GET	/api/v1/cart/detail	—	CartLine[]	查看购物车（详情）
POST	/api/v1/cart/items	{productId, quantity}	CartLine[]	加入购物车
POST	/api/v1/cart/items/{productId}	{quantity}	—	修改数量
POST	/api/v1/cart/items/{productId}/delete	—	—	移除商品
POST	/api/v1/cart/clear	—	—	清空购物车

订单与支付接口（需 USER 角色）：

方法	路径	请求体/参数	响应 data	说明
POST	/api/v1/orders/delivery	{addressId}	Order	创建外卖订单
POST	/api/v1/orders/group-buy	{dealId, quantity}	Order	创建团购订单
POST	/api/v1/payments	{orderId, clientRequestId}	Order	模拟支付
POST	/api/v1/orders/{id}/cancel	—	Order	取消订单
GET	/api/v1/orders	—	Order[]	订单列表

评价接口（需 USER 角色）：

方法	路径	请求体/参数	响应 data	说明
POST	/api/v1/reviews	{orderId, overallScore, tasteScore, serviceScore, content}	Review	提交评价

商家后台接口（需 MERCHANT_ADMIN 角色）：

方法	路径	请求体/参数	响应 data	说明
GET	/api/v1/merchant-admin/orders	—	Order[]	商家订单列表
POST	/api/v1/merchant-admin/orders/{id}/transition	{nextStatus}	Order	订单状态流转
POST	/api/v1/merchant-admin/coupons/verify	{code}	—	核销券码
GET	/api/v1/merchant-admin/products	—	Product[]	商品列表
POST	/api/v1/merchant-admin/products	{id, name, description, priceCent, stock}	Product	新增/更新商品
POST	/api/v1/merchant-admin/products/{id}/toggle	—	Product	上下架商品

方法	路径	请求体/参数	响应 data	说明
GET	/api/v1/merchant-admin/group-deals	—	GroupDeal[]	团购列表
POST	/api/v1/merchant-admin/group-deals	{id, title, description, priceCent, stock}	GroupDeal	新增/更新团购
POST	/api/v1/merchant-admin/group-deals/{id}/toggle	—	GroupDeal	启停团购

平台管理接口（需 PLATFORM_ADMIN 角色）：

方法	路径	请求体/参数	响应 data	说明
GET	/api/v1/admin/metrics	—	{userCount, merchantCount, orderCount, totalRevenueCents, health, logs}	平台指标

AI 客服接口（无需鉴权）：

方法	路径	请求体/参数	响应 data	说明
POST	/api/v1/assistant/ask	{question}	{answer}	智能问答

4.2 内部界面设计

4.2.1 后端模块间调用关系

系统后端采用单服务架构，所有业务逻辑集中在 `DemoStore` 类中。模块间的调用关系体现在方法调用上：

调用方模块	被调用方模块	调用场景
订单模块 (createDeliveryOrder)	购物车模块 (cartDetail)	读取购物车生成订单
订单模块 (createDeliveryOrder)	商品模块 (products)	校验商品状态和价格
订单模块 (createGroupOrder)	团购模块 (deals)	校验团购套餐有效性
支付模块 (pay)	订单模块 (orders)	更新订单支付状态
支付模块 (pay)	券码模块 (couponCode)	团购支付后生成券码
评价模块 (review)	订单模块 (orders)	校验订单完成状态和防重复评价
评价模块 (review)	商家模块 (merchants)	更新商家评分统计
券码模块 (verifyCoupon)	订单模块 (orders)	查找券码对应订单并更新状态
商家管理模块 (transition)	订单模块 (orders)	推进订单状态流转

4.2.2 前端组件调用关系

前端采用单文件组件架构，所有组件定义在 `main.tsx` 中。`App` 组件作为根组件持有全局状态，各子组件通过 `props` 接收数据和回调函数：

```
App (根组件)
├─ Login (登录/注册表单)
├─ Home (首页：搜索栏 + 分类筛选 + 商家卡片)
├─ Detail (商家详情：商品列表 + 团购 + 评价)
├─ Cart (购物车：商品行 + 地址选择 + 提交)
├─ Orders (订单列表：支付/取消/评价操作)
│   └─ StatusTimeline (订单状态时间线)
├─ Profile (个人信息：昵称编辑 + 地址管理)
├─ MerchantDesk (商家工作台：履约 + 核销 + 商品管理)
├─ Admin (平台看板：KPI 卡片 + 图表 + 日志)
└─ Assistant (AI 客服：问答交互)
```

组件数据流：

- App 持有 `user`, `view`, `keyword`, `categories`, `merchants`, `activeMerchant`, `orders`, `cart`, `addresses`, `selectedAddressId`, `message` 等状态
- 子组件通过 props 接收数据，通过回调函数（如 `onLogin`, `openMerchant`, `addCart`, `pay`, `cancelOrder`）向 App 反馈操作
- 所有 API 调用在 App 中的回调函数内完成，子组件不直接调用 `api()` 函数

4.3 用户界面设计

4.3.1 设计规范

系统采用青绿色调（CSS 变量主题），适配桌面和移动端浏览器。

设计要素	规范
主色调	<code>--teal: #0d9488</code> （青绿，品牌主色）
辅色调	<code>--coral: #f97316</code> （珊瑚橙，强调色）
背景色	<code>--cream: #faf7f2</code> （米白，页面背景）
字体	系统默认中文字体（微软雅黑/PingFang SC）
圆角	卡片 12px, 按钮 8px
阴影	卡片 <code>0 2px 12px rgba(0,0,0,0.06)</code>
图标	Lucide React 图标库
布局	左侧边栏(200px) + 右侧内容区

4.3.2 页面详细设计

登录页 (/login) :

- 居中卡片布局（400px 宽）
- 登录/注册模式切换（分段控件）
- 登录模式：手机号输入框 + 密码输入框 + 登录按钮
- 注册模式：增加昵称输入框 + 注册按钮
- 底部预设演示账号快捷填充

首页 (/home) :

- 顶部搜索栏：文本输入框 + 搜索按钮
- 分类标签行：水平排列的分类筛选按钮（全部 + 各分类）

- 商家卡片网格 (2-3列自适应)：每个卡片含封面图、名称、分类标签、距离、评分 (★星标)、人均价格、“推荐理由”标签
- 空状态：无搜索结果时显示引导文字

商家详情页 (/detail/{id})：

- 顶部 Hero 区：商家封面大图 + 名称 + 地址 + 推荐理由
- 双栏布局：左侧商品列表 (名称、描述、价格、“加入购物车”按钮)，右侧团购套餐列表 + 评价列表
- 底部“去购物车”悬浮按钮

购物车页 (/cart)：

- 空状态插图和提示
- 商品行：名称 + 单价 + 数量调节器 (-/+按钮) + 小计 + 删除按钮
- 清空购物车按钮
- 结算区：收货地址单选列表 + 总额显示 + “提交订单”按钮
- 无地址时提示先添加地址

订单页 (/orders)：

- 空状态提示
- 订单卡片：订单号、类型标签 (外卖/团购)、状态标签 (颜色区分)、商品行列表、总额、状态时间线
- 团购已支付订单展示12位券码 (可复制)
- 操作按钮：待支付→“去支付”+“取消”；已完成/已使用未评价→“评价”
- StatusTimeline 子组件：横向时间线，每节点显示状态名和时间

个人信息页 (/profile)：

- 双栏布局：左侧资料编辑 + 地址列表，右侧新增地址表单
- 头像预览 (圆形缩略图)
- 地址卡片：联系人、电话、详细地址、默认标签、设为默认/删除按钮

商家工作台 (/merchant)：

- 四个功能面板 (可折叠或Tab切换)：
 1. 订单履约：订单列表 + “下一步”按钮 (接单→配送→完成)
 2. 券码核销：输入框 + 核销按钮
 3. 商品维护：新增表单 (名称、描述、价格、库存) + 商品列表 (含上架/下架开关)
 4. 团购维护：新增表单 + 团购列表 (含启用/停用开关)

平台看板 (/admin) :

- 4个KPI卡片（横向排列）：用户总数、商家总数、订单总数、交易总额
- Recharts 柱状图可视化
- 操作日志列表（最近8条）

AI客服页 (/assistant) :

- 问题输入框 + 发送按钮
- 预置示例问题
- 回答展示区（卡片样式）

4.3.3 错误提示设计

- 表单校验错误：输入框下方红色文字提示
- 接口业务错误：页面顶部 Toast 通知（3秒自动消失）
- 网络错误：Toast 显示"网络异常，请稍后重试"
- 401 鉴权失败：自动跳转至登录页

5 异常处理设计

5.1 出错信息管理

5.1.1 后端异常分类与处理

系统通过 `BusinessException`（自定义运行时异常）和 `GlobalExceptionHandler`（全局异常处理器）实现统一的错误处理。

`BusinessException` 错误码体系：

错误码范围	HTTP状态码	含义	示例
40000	400 BAD_REQUEST	参数校验失败或业务规则限制	"手机号已注册""最多添加5个地址""购物车为空"
40100	401 UNAUTHORIZED	认证失败	"密码错误""用户不存在"
40300	403 FORBIDDEN	权限不足	"无权操作其他商家的商品""无权访问此订单"
40400	404 NOT_FOUND	资源不存在	"商品不存在""订单不存在""地址不存在"
40900	409 CONFLICT	业务冲突	"订单状态不允许此操作""该订单已评价"
50000	500 INTERNAL_SERVER_ERROR	系统内部异常	未预期的 RuntimeException

GlobalExceptionHandler 处理逻辑：

```
@ExceptionHandler(BusinessException.class)
public ApiResponse<Void> handleBusiness(BusinessException e) {
    int httpStatus = switch (e.getCode() / 100) {
        case 400 -> 400;
        case 401 -> 401;
        case 403 -> 403;
        case 404 -> 404;
        default -> 409;
    };
    // 设置 HTTP 响应状态码，返回 ApiResponse.fail(e.getCode(), e.getMessage())
}

@ExceptionHandler(Exception.class)
public ApiResponse<Void> handleUnknown(Exception e) {
    // 返回 500，日志记录完整堆栈，响应用户"系统异常，请稍后重试"
}
```

5.1.2 前端错误处理

前端 `api<T>()` 通用函数统一处理 HTTP 错误：

```
async function api<T>(path: string, options: RequestInit = {}): Promise<T> {
  const response = await fetch(path, { ... });
  const json = await response.json();
  if (json.code !== 200) throw new Error(json.message);
  return json.data;
}
```

- 每次 API 调用由 try-catch 包裹，捕获错误后设置 `message` 状态显示 Toast
- 401 错误 (Token 失效)：清除 localStorage 中的 Token，设置 view 为 "login"

5.2 故障预防与补救

5.2.1 支付幂等性保障

- 每次支付请求携带唯一的 `clientRequestId` (前端使用 `crypto.randomUUID()` 生成)
- 后端维护 `Map<String, Long>` 记录已处理的 `clientRequestId`
- 重复支付请求直接返回已有结果，不重复扣款

5.2.2 订单事务一致性

- 创建订单时，订单主表写入和购物车清空在同一方法内顺序执行
- 支付时，支付记录创建和订单状态更新在 `pay()` 方法内原子完成
- 评价时，评价记录创建和商家评分重算同步完成

5.2.3 数据完整性

- 用户密码仅保存 BCrypt 哈希值，永不返回给前端
- 手机号作为唯一标识，注册时检查重复
- 评价接口校验订单归属，防止越权评价

5.2.4 状态机保护

订单状态流转通过严格的状态机校验防止非法操作：

PENDING_PAYMENT → PAID	(支付成功)
PENDING_PAYMENT → CANCELLED	(用户取消)
PAID → ACCEPTED	(商家接单, 仅外卖)
ACCEPTED → DELIVERING	(商家配送)
DELIVERING → COMPLETED	(配送完成)
PAID → USED	(团购核销, 仅团购)
COMPLETED → (可评价)	
USED → (可评价)	

任何不符合以上规则的流转将抛出 `BusinessException(40900, "不允许的状态变更")`。

5.3 系统维护设计

5.3.1 健康检查

- Spring Boot Actuator 暴露 `/actuator/health` 和 `/actuator/info` 端点
- 健康检查无需鉴权, 返回 `{"status": "UP"}`
- 可用于开发调试和部署验证

5.3.2 操作日志

- 平台管理员可在看板查看操作日志
- 日志记录格式: `{id, actor, action, createdAt}`
- 关键操作 (登录、下单、支付、核销) 自动记录

5.3.3 数据重置

- 系统采用内存存储, 重启后自动恢复为种子数据
- 无需手动清理或迁移数据
- 适合课程演示场景的快速重置

6 性能优化和安全设计

6.1 性能优化设计

6.1.1 前端优化

优化策略	实现方式
构建工具	Vite 提供快速的 ESBuild 打包和 HMR 热更新
代码分割	TypeScript 编译时类型检查，减少运行时错误
状态管理	使用 React useState 组件级状态，避免全局状态库的额外开销
条件渲染	仅渲染当前 view 对应的组件，其余组件不挂载 DOM
CSS 优化	使用 CSS 变量减少重复样式声明，单文件 99 行覆盖全部样式

6.1.2 后端优化

优化策略	实现方式
内存存储	所有数据在内存中（ConcurrentHashMap），读写 O(1)
无数据库连接池	无 JDBC/ORM 开销，适合演示场景的低延迟要求
搜索过滤	使用 Java Stream API 进行内存过滤和排序，数据集不超过千级
分页支持	商家列表支持分页，避免一次返回全量数据
响应时间	所有接口均在 100ms 内响应（无网络IO），远超 2-3 秒的性能目标

6.1.3 后端性能指标实测估计

操作	估计响应时间	目标时间	达标
登录/注册	<50ms	2秒	远优于目标
商家列表	<100ms	3秒	远优于目标
关键词搜索	<50ms	2秒	远优于目标
购物车更新	<30ms	1秒	远优于目标
创建订单	<50ms	3秒	远优于目标
模拟支付	<50ms	3秒	远优于目标

6.2 安全设计

6.2.1 认证与鉴权

安全机制	实现方式
密码加密	BCryptPasswordEncoder (rounds=10) 对密码做哈希存储
Token 认证	服务端生成 UUID Token, 客户端通过 <code>Authorization: Bearer <token></code> 携带
角色鉴权	Spring Security 基于角色的访问控制: USER / MERCHANT_ADMIN / PLATFORM_ADMIN
接口保护	受保护接口必须携带有效 Token, 否则返回 401
CSRF	前后端分离架构下已禁用 CSRF 防护 (无 Session)

6.2.2 权限控制详情

接口路径前缀	所需角色	说明
<code>/api/v1/auth/**</code>	无需鉴权	登录、注册
<code>/api/v1/merchants/**</code>	无需鉴权	商家浏览、搜索
<code>/api/v1/categories</code>	无需鉴权	分类查询
<code>/api/v1/actuator/**</code>	无需鉴权	健康检查
<code>/api/v1/user/**</code>	USER	个人信息、地址管理
<code>/api/v1/cart/**</code>	USER	购物车操作
<code>/api/v1/orders/**</code>	USER	订单操作
<code>/api/v1/payments</code>	USER	支付操作
<code>/api/v1/reviews</code>	USER	评价操作
<code>/api/v1/merchant-admin/**</code>	MERCHANT_ADMIN	商家后台管理
<code>/api/v1/admin/**</code>	PLATFORM_ADMIN	平台管理

6.2.3 数据安全

安全措施	实现方式
密码不返回	<code>safeUser()</code> 方法返回用户信息前剥离 <code>password</code> 字段
权限校验	商家操作校验 <code>order.merchantId == admin.merchantId</code> ，防止跨商家操作
输入校验	手机号格式校验、评分范围 1-5、地址数量上限 5
金额精度	统一以"分"(cents)为单位，使用 <code>long</code> 类型避免浮点精度损失
日志脱敏	操作日志不记录密码、手机号等敏感信息
CORS	开发环境通过 Vite proxy 代理请求，生产环境由同源部署规避跨域问题

6.2.4 Spring Security 配置

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**", "/api/v1/merchants/**",
                "/api/v1/categories", "/actuator/**",
                "/swagger-ui/**", "/v3/api-docs/**").permitAll()
            .requestMatchers("/api/v1/admin/**").hasRole("PLATFORM_ADMIN")
            .requestMatchers("/api/v1/merchant-admin/**").hasRole("MERCHANT_ADMIN")
            .requestMatchers("/api/v1/**").authenticated()
            .anyRequest().permitAll()
        )
        .addFilterBefore(tokenFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

TokenFilter 从 Authorization Header 提取 Bearer Token，调用 `DemoStore.userByToken()` 查找用户，将 User 对象和角色信息设置到 SecurityContextHolder。

7 项目测试计划

7.1 测试策略

本系统采用自底向上的测试策略：先进行后端单元测试，再进行前后端集成测试，最后进行系统级别的端到端测试。测试环境无需数据库和外部服务，所有测试基于内存数据运行。

7.2 测试类型与范围

序号	测试类型	对应部分	测试内容	计划测试时间
1	单元测试	后端 DemoStore	核心业务规则验证	编码阶段
2	单元测试	前端组件	各组件渲染和交互	编码阶段
3	集成测试	API 接口	接口请求/响应正确性	联调阶段
4	集成测试	前后端联调	数据流完整性	联调阶段
5	系统测试	用户端流程	注册→浏览→下单→支付→评价 完整链路	验收阶段
6	系统测试	商家端流程	接单→配送→核销 完整链路	验收阶段
7	系统测试	平台管理	指标看板数据准确性	验收阶段
8	安全测试	鉴权与权限	未登录访问、越权操作	验收阶段

7.3 后端单元测试用例

系统已编写 JUnit 5 测试类 `DemoStoreTest`，覆盖 10 个核心测试用例：

用例编号	测试方法	覆盖规则
UT-01	paymentIsIdempotentForSameClientRequestId	同一 clientRequestId 多次支付，结果一致
UT-02	normalUserLoginCanReturnNullMerchantId	普通用户登录 merchantId 为 null
UT-03	illegalMerchantStateTransitionIsRejected	非法状态流转被拒绝
UT-04	userCanOnlyKeepFiveAddresses	地址数量上限 5
UT-05	merchantCannotToggleOtherMerchantProduct	跨商家操作被拒绝
UT-06	userCanCancelOnlyPendingPaymentOrder	仅待支付订单可取消
UT-07	cartItemQuantityCanBeChangedAndRemoved	购物车增删改查
UT-08	registerCreatesNormalUserWithEncodedPassword	注册后密码已哈希
UT-09	deliveryOrderUsesSelectedAddressSnapshot	订单保存地址快照
UT-10	deliveryOrderRequiresAnAddress	外卖订单必须选择地址

7.4 集成测试计划

用例编号	接口	测试内容	预期结果
IT-01	POST /api/v1/auth/login	正确密码登录	返回 Token 和 User
IT-02	POST /api/v1/auth/login	错误密码登录	返回 401, 提示"密码错误"
IT-03	POST /api/v1/auth/register	重复手机号注册	返回 400, 提示"手机号已注册"
IT-04	GET /api/v1/merchants	关键词搜索"川"	返回名称含"川"的商家
IT-05	GET /api/v1/merchants	按分类筛选	返回对应分类的商家
IT-06	GET /api/v1/merchants/{id}	查看商家详情	返回商家+商品+团购+评价
IT-07	POST /api/v1/cart/items	加入购物车	购物车新增商品
IT-08	POST /api/v1/cart/items	跨商家加购	清空旧购物车后添加新商品
IT-09	POST /api/v1/orders/delivery	外卖下单	返回 PENDING_PAYMENT 订单
IT-10	POST /api/v1/payments	模拟支付	订单状态变为 PAID
IT-11	POST /api/v1/payments	重复支付	幂等返回已有结果
IT-12	POST /api/v1/orders/{id}/cancel	取消待支付订单	状态变为 CANCELLED
IT-13	POST /api/v1/reviews	评价已完成订单	返回评价记录, 商家评分更新
IT-14	POST /api/v1/reviews	重复评价	返回 409
IT-15	POST /api/v1/merchant-admin/orders/{id}/transition	商家接单	状态 PAID→ACCEPTED
IT-16	POST /api/v1/merchant-admin/coupons/verify	核销券码	状态 PAID→USED

用例编号	接口	测试内容	预期结果
IT-17	GET /api/v1/admin/metrics	平台管理员查看看板	返回统计数据
IT-18	GET /api/v1/admin/metrics	普通用户访问	返回 403

7.5 系统测试计划

用例编号	测试场景	操作步骤	预期结果
ST-01	用户完整外卖流程	登录→搜索商家→查看详情→加购→下单→支付→查看订单	订单状态按时间线正确更新
ST-02	商家完整履约流程	商家登录→查看订单→接单→配送→完成	订单状态流转至 COMPLETED
ST-03	团购完整流程	用户购买团购→支付→获得券码→商家输入券码核销	订单状态变为 USED
ST-04	评价流程	订单完成后提交评价→查看商家评分变化	评价记录新增，评分更新
ST-05	取消订单流程	创建订单→不支付→取消订单	订单状态变为 CANCELLED
ST-06	平台管理流程	管理员登录→查看看板	KPI 数据与系统状态一致
ST-07	未登录访问	清除 Token→访问购物车接口	返回 401
ST-08	越权操作	商家A管理员操作商家B订单	返回 403
ST-09	地址上限	连续添加6个地址	第6个返回 400 错误提示

7.6 安全测试计划

用例编号	测试内容	预期结果
SEC-01	无 Token 访问受保护接口	返回 401
SEC-02	伪造 Token 访问受保护接口	返回 401
SEC-03	普通用户访问商家后台接口	返回 403
SEC-04	商家A访问商家B的订单	返回 403
SEC-05	普通用户访问平台管理接口	返回 403
SEC-06	密码在 API 响应中不出现	所有用户相关接口不含 password 字段